



Métodos de arreglos



¿Sabes que son los
métodos de arreglos?
¡Escríbelo en el chat!



Recursos asincrónicos

¿Revisaste los recursos de la semana 5?

- *Presentación de clases*
- *Guía - Métodos de los arreglos*
- *Guía de ejercicios - Modificando elementos de un arreglo: El método find*
- *Desafío – To-do list*

¿Tienes dudas sobre el material o algo revisado en ellos?



/*Agregar un elemento a un arreglo*/

Agregar al final con push

Para agregar un elemento a un arreglo en JavaScript, puedes utilizar el método `push()`. Este método agrega uno o más elementos al final del arreglo y devuelve la nueva longitud del arreglo. Aquí tienes un ejemplo básico.

```
let miArreglo = [1, 2, 3];  
  
// Agregar un elemento al final del arreglo  
miArreglo.push(4);  
  
console.log(miArreglo); // Output: [1, 2, 3, 4]
```

En este caso, hemos agregado el número 4 al final del arreglo `miArreglo` utilizando el método `push()`.

Agregar al principio con unshift

```
let miArreglo = [2, 3, 4];

// Agregar elementos al principio del arreglo
miArreglo.unshift(1);

console.log(miArreglo); // Output: [1, 2, 3, 4]

// Agregar múltiples elementos al principio del arreglo
miArreglo.unshift(-2, -1, 0);

console.log(miArreglo); // Output: [-2, -1, 0, 1, 2, 3, 4]
```

El método `unshift()` se utiliza para agregar uno o más elementos al principio de un arreglo en JavaScript. Aquí te dejo un ejemplo.

En este ejemplo, el método `unshift()` agrega el número 1 al principio del arreglo `miArreglo`. Luego, se agrega un conjunto de números `(-2, -1, 0)` al principio del arreglo.

Es importante mencionar que, al contrario de `push()`, el método `unshift()` devuelve la nueva longitud del arreglo después de la adición de elementos. Por lo tanto, si necesitas saber la longitud actual del arreglo después de utilizar `unshift()`, puedes almacenar ese valor en una variable:

```
let nuevaLongitud = miArreglo.unshift(-2, -1, 0);  
console.log(nuevaLongitud); // Output: 7  
console.log(miArreglo); // Output: [-2, -1, 0, 1, 2, 3, 4]
```

En este caso, `nuevaLongitud` contendría el valor 7, que es la longitud actual del arreglo después de agregar elementos al principio.

/*Eliminando elementos de un arreglo*/

Método splice

El método `splice()` es un método versátil que se utiliza en JavaScript para modificar el contenido de un arreglo, eliminando o reemplazando elementos existentes y/o agregando nuevos elementos en una posición específica del arreglo.

La sintaxis básica del método `splice()` es la siguiente:

```
array.splice(start, deleteCount, item1, item2, ...);
```

- **start**: Es el índice en el que comienza la modificación del arreglo.
- **deleteCount**: Es el número de elementos que se eliminarán a partir del índice **start**.
- **item1, item2, ...**: Son los elementos que se agregarán al arreglo, comenzando desde el índice **start**.

El método findIndex

el método `findIndex()` se utiliza en JavaScript para encontrar el índice de un elemento en un arreglo que cumple con una condición proporcionada en forma de una función de retorno de llamada (callback). Este método devuelve el índice del primer elemento que satisface la condición o -1 si no se encuentra ningún elemento que cumpla con la condición.

Aquí tienes un ejemplo de cómo se utiliza `findIndex()`:

```
let miArreglo = [10, 20, 30, 40, 50];

// Encontrar el índice del primer elemento mayor que 25
let indice = miArreglo.findIndex(function(elemento) {
  return elemento > 25;
});

console.log(indice); // Output: 2
```

Trabajando con objetos

En lugar de solo guardar strings guardaremos objetos con un identificador.

```
const invitados = [  
  { id: 431, nombre: "Jhonny Depp" },  
  { id: 124, nombre: "Brad Pitt" },  
  { id: 541, nombre: "Leonardo DiCaprio" },  
  { id: 664, nombre: "Morgan Freeman" }  
];
```

El identificador tiene que ser único, usualmente este valor lo entrega el backend cuando insertamos el valor. Por ahora, generaremos estos valores localmente.

Modificamos nuestro código inicial

Insertamos un objeto en lugar de un string, el ID lo generamos automáticamente, podríamos ocupar un contador, pero para este caso es más sencillo obtener el tiempo en segundos desde 1970, de esta forma es un identificador único para el contexto del ejercicio.

```
btnAgregar.addEventListener("click", () => {  
  /* Agregamos el invitado al arreglo */  
  nuevoInvitado = {id: Date.now(), nombre: invitadoInput.value}  
  invitados.push(nuevoInvitado)  
  invitadoInput.value = ""  
  
  /* Actualizamos la información en el HTML */  
  let html = ""  
  for (invitado of invitados) {  
    html += `<li>${invitado.nombre}</li>`  
  }  
  listaDeInvitados.innerHTML = html;  
})
```

{desafío}
latam_



Tenemos que mostrar el nombre del invitado, no el objeto.

Implementando el borrado

```
btnAgregar.addEventListener("click", () => {  
  /* Agregamos el invitado al arreglo */  
  nuevoInvitado = {id: Date.now(), nombre: invitadoInput.value}  
  invitados.push(nuevoInvitado)  
  invitadoInput.value = ""  
  
  /* Actualizamos la información en el HTML */  
  let html = ""  
  for (invitado of invitados) {  
    html += `- ${invitado.nombre} <button onclick="borrar(${invitado.id})"> x  
</button> </li>`;   
  }  
  listaDeInvitados.innerHTML = html;  
})

```

{desafío}
latam_



Agregamos un botón que llame a la función borrar y le pasamos el ID

Implementando el borrado

```
function borrar(id){  
  const index = invitados.findIndex((ele) => ele.id == id)  
  invitados.splice(index, 1)  
  
  /* Actualizamos la información en el HTML */  
  let html = ""  
  for (invitado of invitados) {  
    html += `<li>${invitado.nombre} <button onclick="borrar(${invitado.id})"> x  
</button> </li>`;   
  }  
  listaDeInvitados.innerHTML = html;  
}
```



Creamos la función borrar, donde buscamos el elemento por su id y luego lo borramos ocupando splice, finalmente actualizamos la página.

Reutilizando código

Creando la función render

```
/* Actualizamos la información en el HTML
*/
function renderInvitados(){
  let html = ""
  for (invitado of invitados) {
    html += `<li>${invitado.nombre} <button
onclick="borrar(${invitado.id})"> x </button>
</li>`;
  }
  listaDeInvitados.innerHTML = html;
}
```

Como será frecuente que tengamos que actualizar la lista de invitados, por ejemplo al agregar un invitado nuevo, al borrar uno, al modificar uno, al agregar los valores iniciales, es mejor crear una función y llamar cuando se necesiten actualizar.

```
function borrar(id){
  const index = invitados.findIndex((ele) => ele.id == id)
  invitados.splice(index, 1)
  renderInvitados()
}
```

Ejercicio: Servicios de hotelería



Contexto

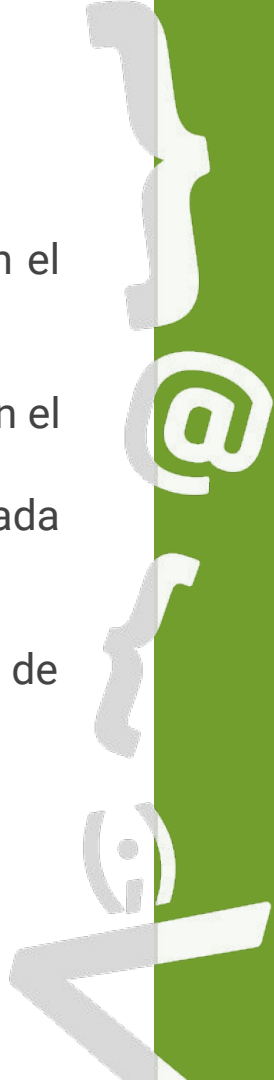
Eres un desarrollador web y te han encargado desarrollar una aplicación para gestionar los servicios adicionales ofrecidos por un hotel a sus huéspedes. La aplicación debe permitir a los usuarios eliminar servicios de la lista, como servicio a la habitación, acceso al spa, entre otros. Crea una interfaz simple que muestre la lista de servicios del hotel y permita a los usuarios eliminar servicios específicos.



Requerimientos

1. Completa el código en app.js para mostrar la lista de servicios del hotel en el HTML.
2. Agrega botones de "Eliminar" para cada servicio en la lista.
3. Implementa la funcionalidad para eliminar un servicio cuando se hace clic en el botón correspondiente.
4. Asegúrate de que la lista se actualice correctamente después de cada eliminación.

Para este desarrollo del ejercicio, utiliza las siguientes sugerencias de HTML y de arreglos.



```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gestión de Servicios de Hotel</title>
  <style>
    body {
      font-family: Arial, sans-serif;
    }

    ul {
      list-style-type: none;
      padding: 0;
    }

    li {
      margin-bottom: 10px;
    }

    button {
      background-color: #dc3545;
      color: white;
      padding: 5px 10px;
      border: none;
      cursor: pointer;
    }
  </style>
</head>
<body>
  <h2>Gestión de Servicios de Hotel</h2>
  <p>Elimina los servicios que no necesitas durante tu estancia:</p>
  <ul id="servicios-lista"></ul>

  <script src="app.js"></script>
</body>
</html>
```

```
const serviciosHotel = [
  { id: 1, nombre: "Desayuno buffet" },
  { id: 2, nombre: "Servicio a la
habitación" },
  { id: 3, nombre: "Acceso al spa" },
  { id: 4, nombre: "Wi-Fi de alta
velocidad" },
  { id: 5, nombre: "Estacionamiento
privado" },
];
```

Revisemos el Desafío: "To-do List"



{desafío}
latam_

*Academia de
talentos digitales*

